

Paradigmas de Programación

Curso Suarez

Trabajo Práctico 1
Programación Orientada a Objetos

24 de septiembre de 2026

Alexander Cruz
109044
acruz@fi.uba.ar

Índice

1. Introducción **3**

2. Objetivo **3**

3. Consigna **3**

4. Alcance **3**

5. Detalles de Implementación **4**

5.1. Polimorfismo con herencia en la jerarquia de Artista 4

5.2. Polimorfismo con herencia en la jerarquia de Ubicación 4

5.3. Polimorfismo para la gestión del medio de pago 4

6. Diagrama de Clases **4**

6.1. Explicacion del Diseño 4

7. Diagramas de Secuencia **7**

7.1. Secuencia: Se calcula La entrada más barata 7

7.2. Secuencia: Se calcula el costo de una entrada 7

8. Manejo de Errores **8**

8.1. Jerarquia de Errores 8

1. Introducción

El presente informe detalla la solución desarrollada para el Trabajo Práctico 1, que tiene como objetivo aplicar los conceptos fundamentales de la programación orientada a objetos (POO) en el lenguaje Pharo. El problema, que consiste en el desarrollo de un sistema de recomendación de futbolistas, fue resuelto utilizando la metodología Test-Driven Development (TDD), un enfoque que garantiza la robustez y funcionalidad del código a través de pruebas unitarias.

La solución se centra en la construcción de un modelo de clases que se adhiere a los principios de la POO, como el encapsulamiento y, de manera crucial, el polimorfismo. Se ha diseñado una arquitectura que permite manejar la lógica de cálculo del valor de los futbolistas y los distintos criterios de búsqueda de manera flexible y extensible, evitando el uso de estructuras de control rígidas.

Adicionalmente, se ha utilizado la herramienta Mutalk para llevar a cabo una prueba de mutación. Este proceso aseguró la calidad de las pruebas unitarias creadas por el alumno, garantizando que el código no solo funciona, sino que está correctamente validado contra posibles errores. El informe incluye los diagramas de clases y de secuencia que documentan el diseño, junto con una explicación detallada de la implementación y los mecanismos de polimorfismo utilizados.

2. Objetivo

Este es un trabajo práctico individual que tiene como objetivo aplicar los conceptos del paradigma de la orientación a objetos vistos hasta ahora en el curso mediante la resolución de un problema con Pharo.

3. Consigna

Se deberá desarrollar un modelo de clases utilizando la metodología TDD. Las especificaciones son un archivo de pruebas SUnit (.st) adjunto a este enunciado y están dadas en forma de pruebas de casos de uso. Se espera que el alumno genere una solución que permita ejecutar exitosamente las pruebas especificadas manteniendo una alta cobertura y evitando la supervivencia de mutantes. Revisar el material Calidad de nuestras pruebas.

4. Alcance

Se deberá desarrollar el modelo completo de la aplicación, sin incluir la interfaz gráfica. Para esto se especifica una API que está descrita por un conjunto de casos de uso. Este conjunto de pruebas está descrito por la clase dada como enunciado.

Para el manejo de errores y situaciones de excepción, se deberán desarrollar y utilizar las excepciones que se crean convenientes, más allá de que en la prueba de integración provistas no necesariamente se manejan excepciones.

Deberán entregarse:

- todas las clases con sus métodos que resuelven el problema, organizados en una o más categorías según criterio del alumno;
- conjunto de pruebas que garanticen la no supervivencia de mutantes;
- conjunto de pruebas que garanticen la no supervivencia de mutantes;
- un archivo "txt"que contenga la pieza de código que permite ejecutar los test de mutación (descrito aquí: pruebas de mutación);

documentación completa del diseño de clases incluyendo diagramas UML de clases y secuencia (cantidades variables acordes al diseño presentado). Se considerarán entregas válidas a aquellas que contengan las partes anteriores. Las que no tengan alguna de ellas no recibirán corrección.

5. Detalles de Implementación

A continuación se presentan varios ejemplos del uso de polimorfismo, dentro de la solucion.

5.1. Polimorfismo con herencia en la jerarquia de Artista

Durante el desarrollo mediante TDD, se identificó la necesidad de tratar a los artistas nacionales e internacionales de manera uniforme a pesar de que ambos calculan sus costos de manera diferente. Se implemento polimorfismo utilizando **herencia** entre la clase abstracta *Artista* y sus subclases *Nacional* e *Internacional*.

Este mecanismo permitio que la clase *Evento* interactue con cualquier *Artista* mediante el mensaje **calcularCosto:** sin conocer su tipo especifico. Al utilizar Herencia se evito el uso de estructuras de contorl *(if)* para distinguir roles y se respeto el principio de **"Tell, don't ask"**, ya que el estado interno del artista es gestionado por el mismo sin necesidad de exponer *getters* a agentes externos.

5.2. Polimorfismo con herencia en la jerarquia de Ubicación

Del mismo modo, se identificó la necesidad de tratar a las ubicaciones PlateaPreferencial, Campo y Vip de manera uniforme a pesar de que los tres calculan sus costos de manera diferente. Se implemento polimorfismo utilizando **herencia** entre la clase abstracta *Ubicacion* y sus subclases *Plate*, *Campo* y *Vip* .

Este mecanismo permitio que la clase *Evento* interactue con cualquier *UbicUbicación* mediante el mensaje **calcularPrecio:** sin conocer su tipo especifico. Al utilizar Herencia se evito el uso de estructuras de contorl *(if)* para distinguir roles y se respeto el principio de **"Tell, don't ask"**, ya que el estado interno de la Ubicación es gestionado por el mismo sin necesidad de exponer *getters* a agentes externos.

5.3. Polimorfismo para la gestión del medio de pago

Otro punto importante donde se aplico polimorfismo fue en el medio de pago que Evento recibia como parametro en el método **entradaMasbarataPagadaCon:**, que podria ser un objeto de tipo *Efectivo* o *Tarjeta*, sin importar el tipo de objeto que reciba *Evento* confia en la respuesta de cada objeto que responde al método **obtenerBeneficio:**.

6. Diagrama de Clases

El diagramade clases detalla las relaciones que existen entre las distintas entidades del sistema.

6.1. Explicacion del Diseño

- **AlgoTek:** Actúa como el punto de entrada, manteniendo una colección de eventos.
- **Evento:** Actua como el corazon del modelo , donde mantiene un solo *Artista*, una colección de *Ubicaciones* y un solo *MedioDePago*.

- **Jerarquia de Clases**: Las clases *Ubicacion*, *Artista* y *MedioDePago* centralizan los datos comunes y delegan las responsabilidades correspondientes mediante sus metodos abstractos.

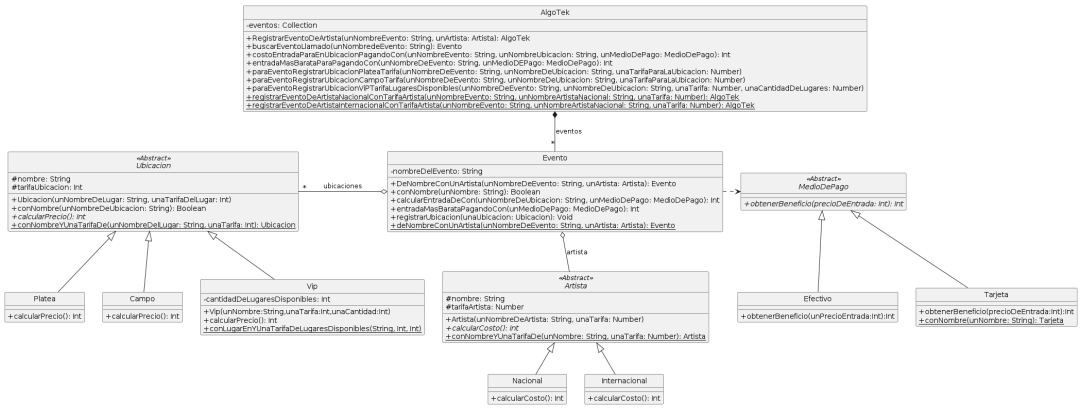
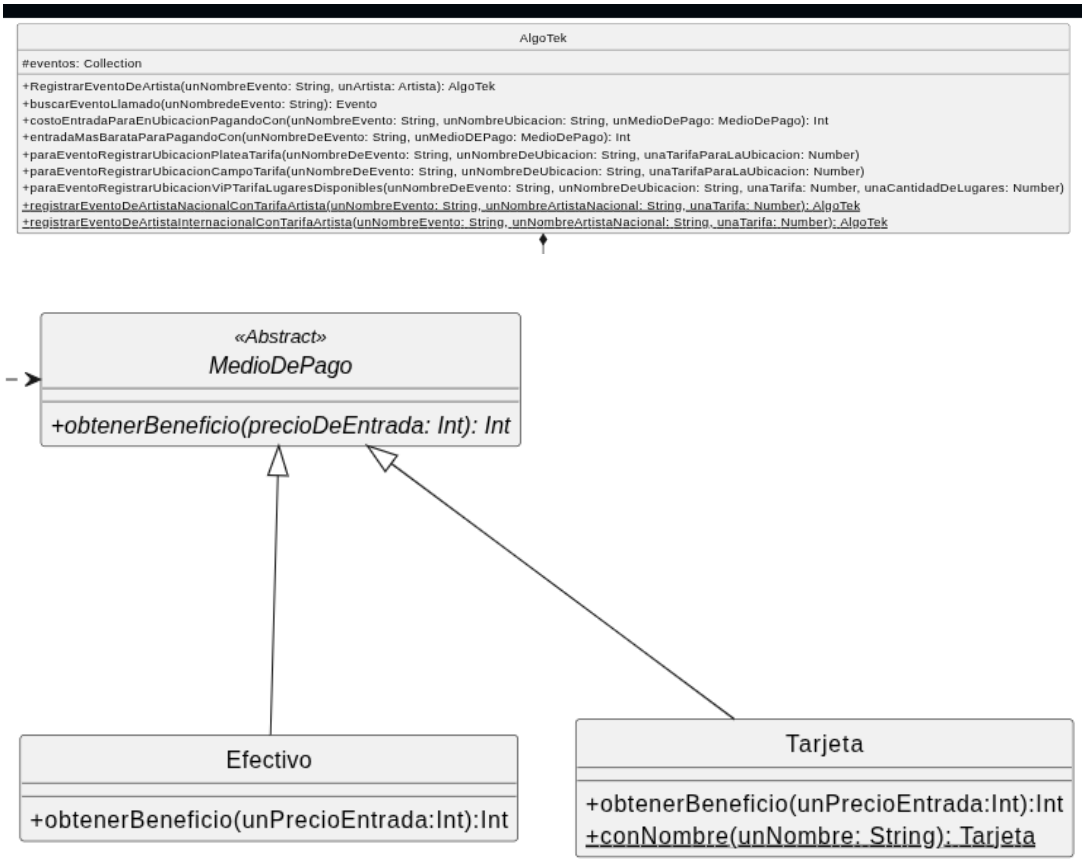
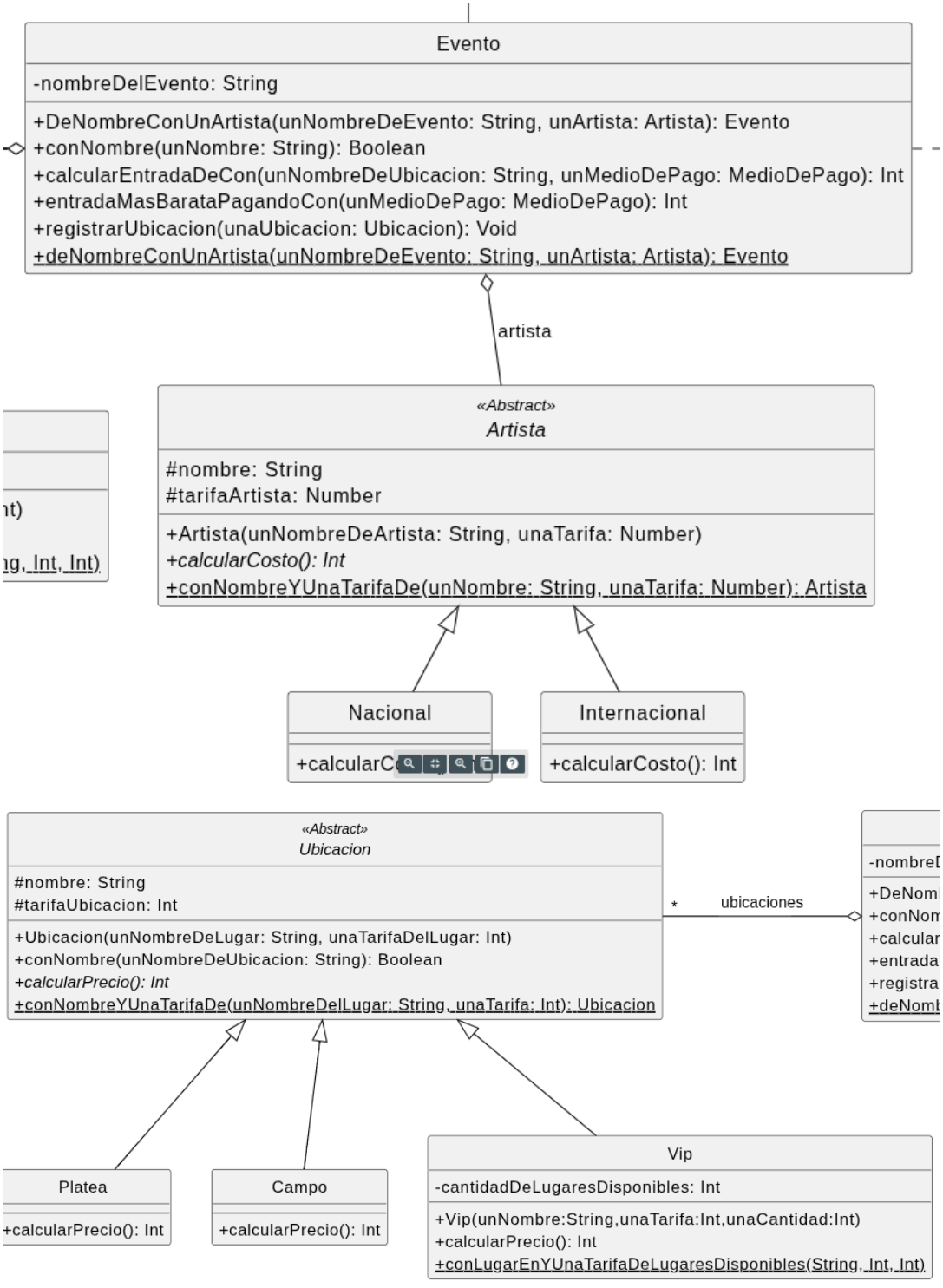
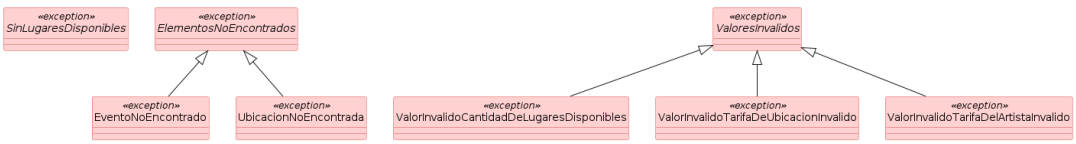


Figura 1: Diagrama de clases: AlgoTek, Evento, Ubicacion, Artista y MedioDePago





Ademaás se incluyen varias clases del tipo exception

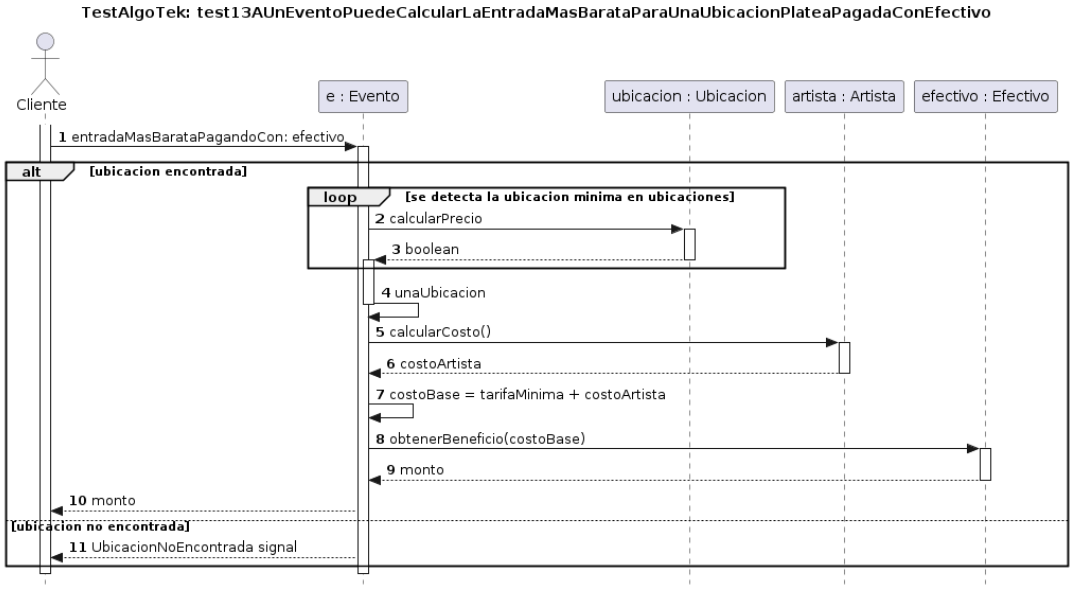


7. Diagramas de Secuencia

Se presentan dos diagramas de secuencia, cada uno corresponde con un test realizado en los CasosDeUso.

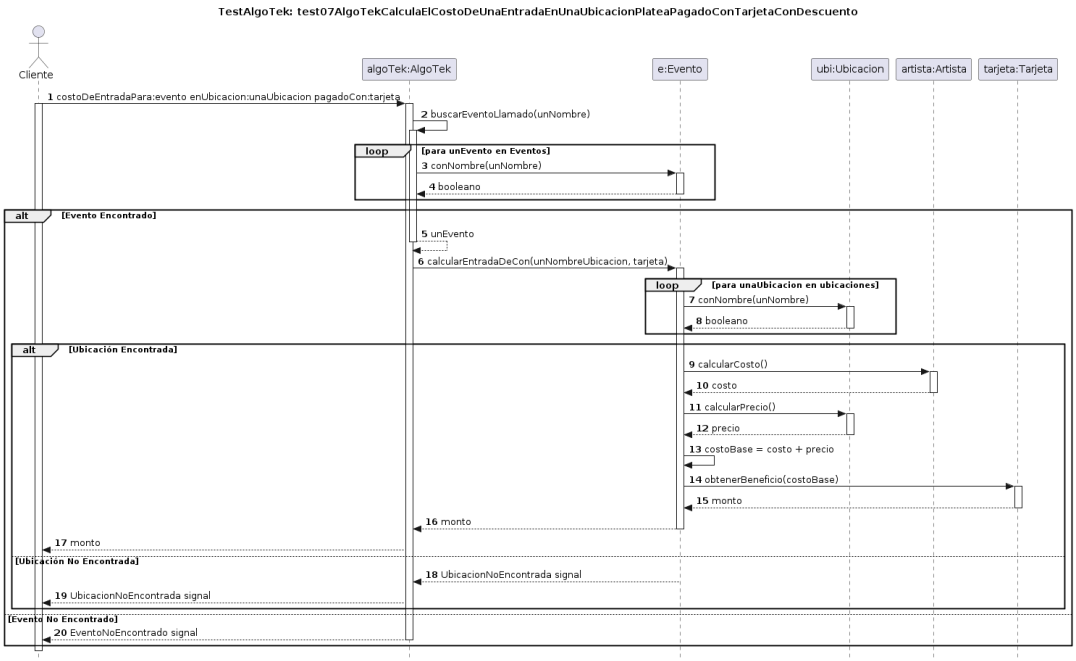
7.1. Secuencia: Se calcula La entrada más barata

Este diagrama de secuencia detalla la ejecución del caso de prueba **test13** de **EventoTEst** Se ilustra el flujo completo desde el Actor(Cliente) solicita la entrada mas barata pagado con efectivo, hasta obtener una valoracion final.



7.2. Secuencia: Se calcula el costo de una entrada

Este diagrama de secuencia corresponde con el el **test07** de **AlgoTekTestPropios** en el cual se detalla el flujo de ejecución desde el Actor(Cliente) solicitando el costo de la entrada para un evento en una Ubicación particular, además se paga con efectivo, la secuencia aorda dos bucles que recorren una coleccion de eventos y otra coleccion de ubicaciones, ademas que se incluye una secuencia del tipo if-Else para ambos Loops.



8. Manejo de Errores

8.1. Jerarquia de Errores

Se diseño una estructura de errore perzonalizadas que heredan de la clase error lo que permite al sistema capturar fallos criticos en la logica de negocio.

- **ElementosNoEncontrado:** Es una clase Base para errores relacionados a elementos que no fueron localizados: **EventoNoEncontrado** y **UbicacionNoEncontrada**.
- **ValoresInvalidos:** Es una Clase Base que esta relacionado con magnitudes fisicas o de conteo **ValorInvalidoCantidadDeLugaresDisponibles**, **ValorInvalidoTarifaDeUbicacionInvalido** y **ValorInvalidoTarifaDelArtistaInvalido**.
- **SinLugaresDisponibles:** Clase Base para errores relacionados cuando la cantidad de lugares disponibles es menor a cero.